

AY: 2025-2026
MIDTERM | ROS 2
Oct. 2025

L3-S5: Dept. of Electrical Engineering
Teacher: A. Mhamdi
Time Limit: 1h

This document contains 10 pages numbered from 1 to 10. Upon receiving it, verify completeness. The 2 tasks are independent and can be solved in any order you prefer. The following rules apply:

- ❶ A handwritten double-sided A4 sheet is permitted.
- ❷ Any electronic material, except basic calculator, is prohibited.
- ❸ Mysterious or unsupported answers will not receive full credit.
- ❹ Express results in scientific notation with 3 sig figs (e.g., $0.01234 \rightarrow 1.23 \times 10^{-2}$).
- ❺ Task N°2: Correct answers earn points as indicated. There is no negative scoring.



Task N°1

⌚ 30mn | (9½ points)

A mobile robot operates in a two-dimensional plane with position coordinates (x, y) . The robot's motion is controlled through a publish-subscribe architecture where velocity commands are transmitted from a publisher node to a subscriber node via a topic named `/cmd_vel`. The subscriber node, which represents the robot controller, computes the robot's updated position based on the received velocity commands. The system is configured as follows:

1. Publisher Node:

The publisher transmits velocity commands using a `geometry_msgs/msg/Twist` message containing:

- `linear.x`: Linear velocity in the x-direction, denoted v_x , measured in meters per second (e.g., $v_x = 0.5$ m/s)
- `angular.z`: Angular velocity about the z-axis, denoted ω_z , measured in radians per second (assumed to be $\omega_z = 0$ for simplicity, constraining motion to be purely translational)

2. Subscriber Node:

Upon receiving a velocity command message, the subscriber updates the robot's position using the discrete-time kinematic equation:

$$x_{\text{new}} = x_{\text{old}} + v_x \cdot \Delta t \quad (1)$$

where:

- x_{new} is the updated position coordinate
- v_x is the linear velocity from `linear.x`
- x_{old} is the previous position coordinate
- Δt is the time step duration

3. Initial Conditions:

The robot begins at the origin $(x_0, y_0) = (0, 0)$.

4. Time Discretization:

The simulation uses a fixed time step of $\Delta t = 1$ second.

(a) Before sending commands, you need to verify the topic configuration.

- ($\frac{1}{2}$ point) Write the **ROS 2** CLI command to list all active topics and verify that `/cmd_vel` exists.

```
ros2 topic list
```

- ($\frac{1}{2}$ point) Write the **ROS 2** CLI command to display the message type used by the `/cmd_vel` topic.

Either one of these works:

```
ros2 topic type /cmd_vel
ros2 topic info /cmd_vel
```

- ($\frac{1}{2}$ point) Write the **ROS 2** CLI command to show the structure/fields of the `geometry_msgs/msg/Twist` message.

```
ros2 interface show geometry_msgs/msg/Twist
```

(b) The robot starts at position $(0, 0)$. You publish a velocity command with $v_x = 0.5$ m/s and $\omega_z = 0$ rad/s.

- ($\frac{1}{2}$ point) Calculate by hand the robot's position after 5 seconds.

With the assumption that $\omega_z = 0$, the robot's motion reduces to straight-line translation along the x-axis. The y-coordinate remains constant:

$$x_{k+1} = x_k + v_x \cdot \Delta t \quad \text{and} \quad y_{k+1} = y_k \quad (2)$$

where k denotes the discrete time index. The new position is therefore:

$$x_{\text{new}} = x_{\text{old}} + v_x \cdot \Delta t$$

$$x_{\text{new}} = 0 + 0.5 \times 5$$

$$x_{\text{new}} = 2.5 \text{ m}$$

Final position: $(2.5, 0)$

- ii. ($\frac{1}{2}$ point) Write the **ROS 2** CLI command to publish this velocity command to the /cmd_vel topic.

```
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.5}}"
```

- (c) The robot is currently at position (3,0). You want to move it backwards (negative x-direction) with $v_x = -0.8$ m/s.

- i. ($\frac{1}{2}$ point) Calculate by hand the robot's position after 4 seconds.

$$x_{\text{new}} = x_{\text{old}} + v_x \cdot \Delta t$$

$$x_{\text{new}} = 3 + (-0.8) \times 4$$

$$x_{\text{new}} = -2 \times 10^{-1} \text{ m}$$

Final position: $(-2 \times 10^{-1}, 0)$

- ii. ($\frac{1}{2}$ point) Write the **ROS 2** CLI command to test this movement using ros2 topic pub.

```
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist "{linear: {x: -0.8}}"
```

- (d) The robot needs to travel from (0,0) to (6,0) meters.

- i. (1 point) If you publish a velocity command with $v_x = 1.2$ m/s, calculate by hand how many seconds it will take to reach the target position.

Using the formula:

$$d = v_x \times t$$

We get:

$$t = \frac{d}{v_x}$$

$$t = 5 \text{ seconds}$$

- ii. (1 point) Write the **ROS 2** CLI command to publish this velocity for a limited duration (one-time publish).

```
ros2 topic pub --once /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 1.2}}"
```

- iii. (1 point) Write the **ROS 2** CLI command to continuously publish this velocity each

0.2 seconds.

For a timestep of 0.2 seconds, the rate would be 5 Hz.

```
ros2 topic pub -r 5 /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 1.2}}"
```

(e) After publishing velocity commands, you want to monitor the system behavior.

- i. (1 point) The robot starts at $(0,0)$ and you publish $v_x = 2.0$ m/s for 3 seconds, then change to $v_x = -0.5$ m/s for 4 seconds. Calculate by hand the final position of the robot.

Step 1: $x_1 = 0 + 2.0 \times 3 = 6.0$ m

Step 2: $x_2 = 6.0 + (-0.5) \times 4 = 6.0 - 2.0 = 4.0$ m

Final position: $(4.0, 0)$

- ii. (1 point) Write the **ROS 2** CLI command to echo/display messages being published on the `/cmd_vel` topic in real-time.

```
ros2 topic echo /cmd_vel
```

- iii. (1 point) Write the **ROS 2** CLI command to check the publishing rate (frequency) of messages on the `/cmd_vel` topic.

```
ros2 topic hz /cmd_vel
```

AY: 2025-2026

Full Name:

L3-S5: Dept. of Electrical Engineering

ID:

MIDTERM | ROS 2

Class: ELNI3

Oct. 2025

Room:

Teacher: A. Mhamdi

Time Limit: 1h

✂

ANSWER SHEET

Task N°2

⌚ 30mn | (10½ points)

- (a) (½ point) What is the output of the following code?

```
x = "Python"
print(x[1:4])
```

☐ Pyt ☒ yth ☐ ytho ☐ yth

- (b) (½ point) What is the result of 3**2 in Python?

☐ 5 ☐ 6 ☒ 9 ☐ 12

- (c) (½ point) Which of the following is the correct way to create a list in Python?

☐ lst = {1, 2} ☐ lst = (1, 2) ☐ lst = <1, 2> ☒ lst = [1, 2]

- (d) (½ point) What is the output of the following code?

```
x = [1, 2, 3]
y = x
y.append(4)
print(x)
```

☒ [1, 2, 3, 4] ☐ [1, 2, 3] ☐ [4] ☐ [1, 2, 4]

- (e) (½ point) Which of the following is used to define a class?

☒ class MyClass: ☐ def MyClass: ☐ class: MyClass

- (f) (½ point) What is an object in Python OOP?

☐ A built-in function
☐ A type of variable
☒ An instance of a class
☐ A module

- (g) (½ point) What is the purpose of the __init__ method in a Python class?

☐ To delete an object from memory

✂

- ☐ To define class methods
- ☐ To create a copy of an object
- ✓ ☒ To initialize the attributes of an object when it is created

(h) ($\frac{1}{2}$ point) What does the `self` parameter represent in Python class methods?

- ✓ ☒ The instance of the class itself
- ☐ The parent class
- ☐ A global variable
- ☐ The class name

(i) ($\frac{1}{2}$ point) What is the output of the following code?

```
class Dog:
    def __init__(self, name):
        self.name = name
d = Dog("Max")
print(d.name)
```

✓ ☒ Max ☐ Dog ☐ name ☐ Error

(j) ($\frac{1}{2}$ point) What is the output of the following code?

```
class Box:
    def __init__(self, length):
        self.length = length
b = Box(10)
print(b.length * 2)
```

☐ 10 ✓ ☒ 20 ☐ 102 ☐ Error

(k) ($\frac{1}{2}$ point) What is the output of the following code?

```
class Counter:
    count = 0
    def __init__(self):
        Counter.count += 1
c1 = Counter()
c2 = Counter()
print(Counter.count)
```

✂

☐ 0 ☐ 1 ☒ 2 ☐ Error

(l) ($\frac{1}{2}$ point) What is the output of the following code?

```
class Animal:
    def speak(self):
        return "Sound"
class Cat(Animal):
    def speak(self):
        return "Meow"
c = Cat()
print(c.speak())
```

☒ Meow ☐ Sound ☐ None ☐ Error

(m) ($\frac{1}{2}$ point) Which concept allows a child class to inherit properties from a parent class?

☐ Encapsulation ☐ Polymorphism ☒ Inheritance ☐ Abstraction

(n) ($\frac{1}{2}$ point) What is method overriding in Python?

- ☐ Creating multiple methods with the same name
☒ Redefining a parent class method in the child class
☐ Deleting a method from a class
☐ Calling a method multiple times

(o) ($\frac{1}{2}$ point) What command is used to run a ROS 2 node from a package?

- ☐ ros2 launch <package_name> <node_name>
☐ ros2 start <package_name> <node_name>
☐ ros2 node run <package_name> <node_name>
☒ ros2 run <package_name> <node_name>

(p) ($\frac{1}{2}$ point) What command is used to start the turtlesim node in ROS 2?

- ☐ ros2 launch turtlesim turtlesim_node
☒ ros2 run turtlesim turtlesim_node
☐ ros2 start turtlesim turtle
☐ ros2 run turtle turtlesim

(q) ($\frac{1}{2}$ point) What is the purpose of a topic in ROS 2?

☒ To allow nodes to exchange messages in a publish-subscribe model

✂

- ☐ To execute Python scripts
 - ☐ To store configuration parameters
 - ☐ To manage node lifecycle
- (r) ($\frac{1}{2}$ point) Which topic is used to control the turtle's movement?
- ✓ `/turtle1/cmd_vel`
 - ☐ `/turtle1/move`
 - ☐ `/turtle1/velocity`
 - ☐ `/turtle1/control`
- (s) ($\frac{1}{2}$ point) What command spawns a new turtle in turtlesim?
- ☐ `ros2 topic pub /spawn turtlesim/msg/Spawn`
 - ☐ `ros2 run turtlesim spawn_turtle`
 - ✓ `ros2 service call /spawn turtlesim/srv/Spawn`
 - ☐ `ros2 call /create_turtle`
- (t) ($\frac{1}{2}$ point) What message type does `/turtle1/cmd_vel` use?
- ☐ `std_msgs/msg/Velocity`
 - ✓ `geometry_msgs/msg/Twist`
 - ☐ `turtlesim/msg/Velocity`
 - ☐ `geometry_msgs/msg/Velocity`
- (u) ($\frac{1}{2}$ point) Which node is used to control turtlesim with keyboard?
- ✓ `turtle_teleop_key`
 - ☐ `turtlesim_keyboard`
 - ☐ `teleop_turtle`
 - ☐ `keyboard_control`

APPENDIX

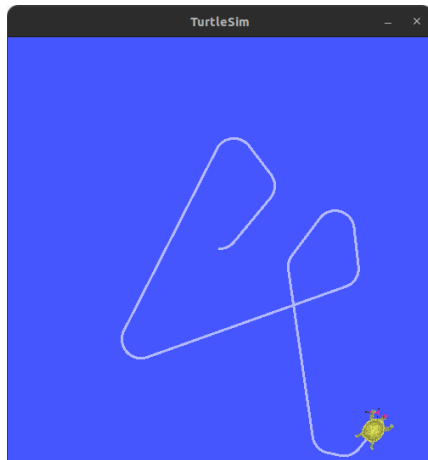


FIG. 1. 'turtlesim_node' Node

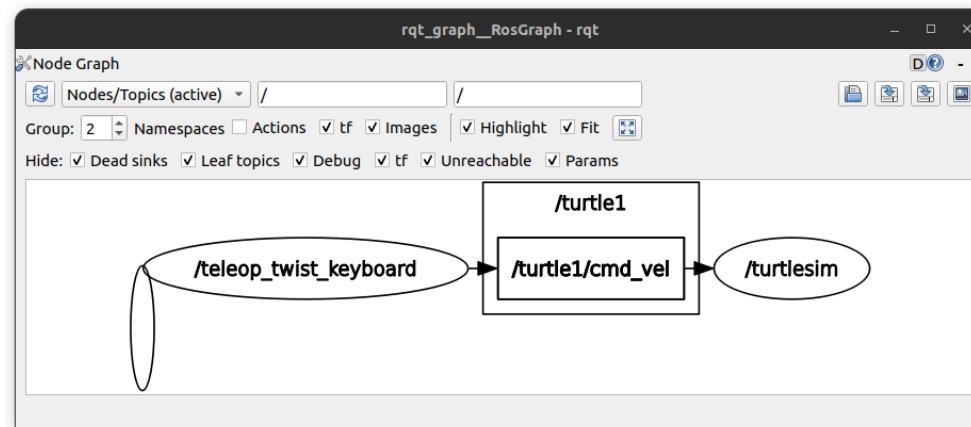


FIG. 2. Nodes and Topics

```

tmux
(venv) >> source /opt/ros/humble/setup.zsh
(venv) >> ros2 pkg executables turtlesim
turtlesim draw_square
turtlesim mimic
turtlesim turtle_teleop_key
turtlesim turtlesim_node
(venv) >> ros2 run turtlesim turtlesim_node
[INFO] [1760276608.133454471] [turtlesim]: Starting turtlesim
with node name /turtlesim
[INFO] [1760276608.138895015] [turtlesim]: Spawning turtle [t
urtle1] at x=[5.544445], y=[5.544445], theta=[0.000000]

(venv) >> ros2 topic list -t
/parameter_events [rcl_interfaces/msg/ParameterEvent]
/rosout [rcl_interfaces/msg/Log]
/turtle1/cmd_vel [geometry_msgs/msg/Twist]
/turtle1/color_sensor [turtlesim/msg/Color]
/turtle1/pose [turtlesim/msg/Pose]
(venv) >> ros2 interface show geometry_msgs/msg/Twist
# This expresses velocity in free space broken into its linea
r and angular parts.

Vector3 linear
  float64 x
  float64 y
  float64 z
Vector3 angular
  float64 x
  float64 y
  float64 z
(venv) >> ros2 topic info /turtle1/cmd_vel
Type: geometry_msgs/msg/Twist
Publisher count: 0
Subscription count: 1
(venv) >>

(venv) >> ros2 topic pub --once /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x:
1., y: 2, z: 0.}, angular: {x: 0., y: 0., z: -1.}}"
publisher: beginning loop
publishing #1: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=1.0, y=2.0, z=
0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=-1.0))

(venv) >> ros2 run teleop_twist_keyboard teleop_twist_keyboard --ros-args --remap /cmd_
vel:=/turtle1/cmd_vel

(venv) >> ros2 service list -t
/clear [std_srvs/srv/Empty]
/kill [turtlesim/srv/Kill]
/reset [std_srvs/srv/Empty]
/spawn [turtlesim/srv/Spawn]
/turtle1/set_pen [turtlesim/srv/SetPen]
/turtle1/teleport_absolute [turtlesim/srv/TeleportAbsolute]
/turtle1/teleport_relative [turtlesim/srv/TeleportRelative]
/turtlesim/describe_parameters [rcl_interfaces/srv/DescribeParameters]
/turtlesim/get_parameter_types [rcl_interfaces/srv/GetParameterTypes]
/turtlesim/get_parameters [rcl_interfaces/srv/GetParameters]
/turtlesim/list_parameters [rcl_interfaces/srv/ListParameters]
/turtlesim/set_parameters [rcl_interfaces/srv/SetParameters]
/turtlesim/set_parameters_atomically [rcl_interfaces/srv/SetParametersAtomically]
(venv) >> ros2 interface show turtlesim/srv/Spawn
float32 x
float32 y
float32 theta
string name # Optional. A unique name will be created and returned if this is empty
---
string name
(venv) >>

(venv) >> ros2 action list -t
/turtle1/rotate_absolute [turtlesim/action/RotateAbsolute]
(venv) >> rqt_graph

```

0 python3 CPU 14.3% GPU AMD RAM 4.6GB/15GB

FIG. 3. ROS 2 CLI